C# Machine Problem 2: Automated Cargo Batch Processing, Manifest Auditing & File Archiving System

Problem Scenario

In Milestone 1, you built a single-package evaluation engine. In an industrial freight terminal and logistics dispatch hub, evaluating packages one at a time and terminating the application is unviable.

The terminal operations director requires a comprehensive, modular console application capable of:

- Processing **batches of multiple cargo parcels** within an active session without termination or state corruption.
- Managing in-memory manifest records dynamically using **arrays and generic `List<T>` collections**.
- Decomposing core business calculations into clean, reusable **modular methods with typed parameters, return types, and C# tuples**.
- Providing an **interactive, loop-driven console menu** featuring strict input validation and loop control keywords (`break`, `continue`).
- **Loading batch data from disk and exporting audited manifests** safely utilizing `File` utilities and the `using` resource-management pattern.
- Organizing the codebase logically across segregated **namespaces**.

Milestone 2 Learning Competencies Assessed

Competency	Application in Machine Problem 2
Loop Control	Continuous interactive menu execution (`do-while` / `while`), parcel iteration (`for` / `foreach`), and stream processing with flow control (`break`, `continue`).
Collections	Dynamic batch accumulation using `List<string>` and `List<decimal>`; fixed tabular arrays for zone rates and header labels.
Modular Methods	Elimination of monolithic code by encapsulating algorithms into static methods with typed parameters, return values, and multi-value tuples.
Scope & Namespaces	Architectural separation between domain calculation engines (`CargoLogistics.Core`) and file/console operations (`CargoLogistics.IO`).
File I/O & Disposal	Persistent reading from `batch_input.txt` and structured writing to `manifest_output.txt` using deterministic resource disposal via `using`.

Namespace Architecture

> **Segregate your project code into two clean namespaces:**
>
> - `CargoLogistics.Core`: Houses calculation engines, decision evaluators, and batch aggregation algorithms.
> - `CargoLogistics.IO`: Houses batch file parser, manifest disk export, and tabular console report generators.

Required Modular Methods

Refactor all operational algorithms out of top-level code into standalone, testable static methods:

Method A: Volumetric & Chargeable Weight Calculator

- **Identifier:** `CalculateChargeableWeight`
- **Namespace:** `CargoLogistics.Core`
- **Signature:** `public static (decimal volumetricWeight, decimal chargeableWeight) CalculateChargeableWeight(decimal physicalKg, int lengthCm, int widthCm, int heightCm)`
- **Logic:** Multiplies length, width, and height to determine volume (cm³). Computes volumetric weight (volume in cm³ / 5000.0m). Returns a named tuple containing both the volumetric weight and the greater of physical vs. volumetric weight.

Method B: Operational Status Evaluator

- **Identifier:** `EvaluateStatus`
- **Namespace:** `CargoLogistics.Core`
- **Signature:** `public static string EvaluateStatus(decimal physicalKg, int lengthCm, int widthCm, int heightCm, int zone, string cargoType, decimal declaredValue, bool hasInvoice, bool hasInsurance)`
- **Return Values:** `"CLEARED FOR DISPATCH"`, `"HELD FOR MANUAL REVIEW"`, or `"SHIPMENT REJECTED"`
- **Decision Hierarchy:** Enforces physical dimensions (≤ 150 cm, volume ≤ 300,000 cm³), weight bounds (0 < kg ≤ 70), hazardous material rules (Zone 3 auto-rejected; Zones 1 & 2 held for review), and customs invoice validation. Rejections take absolute precedence over reviews.

Method C: Fee & Tariff Computation Engine

- **Identifier:** `CalculateFees`
- **Namespace:** `CargoLogistics.Core`
- **Signature:** `public static (decimal baseFee, decimal surcharge, decimal insuranceFee, decimal customsDuty, decimal totalDue) CalculateFees(decimal chargeableWeight, int zone, string cargoType, decimal declaredValue, bool hasInsurance)`
- **Logic:**
  - **Base Fee:** Chargeable weight × zone rate (Zone 1 Domestic: ₱120.00/kg; Zone 2 Regional: ₱350.00/kg; Zone 3 International: ₱780.00/kg).
  - **Cargo Surcharge:** Perishable (+25% of base), Hazardous (+50% of base), High-Value Electronics (+₱1,500.00 flat), General Merchandise (₱0.00).
  - **Insurance Fee:** If requested, 1.5% of declared value or ₱500.00 minimum (whichever is greater); otherwise ₱0.00.
  - **Customs Duty:** Zone 1 is exempt (₱0.00). Zones 2 & 3: ₱0.00 if declared value ≤ ₱10,000.00 (de minimis); 5% on ₱10,000.01 to ₱100,000.00; 12% on value exceeding ₱100,000.00.
  - **Total Due:** Base Fee + Surcharge + Insurance Fee + Customs Duty.

Method D: Batch Analytics Generator

- **Identifier:** `GenerateBatchSummary`
- **Namespace:** `CargoLogistics.Core`
- **Signature:** `public static (int totalPackages, int clearedCount, int heldCount, int rejectedCount, decimal totalRevenue, decimal avgWeight) GenerateBatchSummary(List<decimal> weights, List<decimal> totals, List<string> statuses)`
- **Logic:** Iterates across collection elements using loops to compute total packages, tally counts per status, calculate aggregate revenue, and determine average chargeable weight. Handles empty batch lists cleanly without dividing by zero.

Interactive Console Menu & Operational Flow

The application entry point must run inside a continuous loop (`do-while` or `while`) presenting the operational dispatch menu:

```
============================================================
       CARGO LOGISTICS DISPATCH & AUDIT TERMINAL (M2)
============================================================
  1. Add Single Package to Batch
  2. Load Package Batch from File (batch_input.txt)
  3. View Active Batch Manifest & Summary
  4. Export Audited Manifest to File (manifest_output.txt)
  5. Exit System
============================================================
Enter option (1-5):
```

Menu Option Specifications:

- **Option 1 (Add Single Package):** Interactively prompts the operator for physical weight, dimensions (L, W, H), destination zone (1-3), cargo type, declared value, invoice status, and insurance. Invokes calculation methods, assigns an auto-incremented tracking identifier (e.g., `PKG-1001`), and appends records into active memory collections (`List<string>`, `List<decimal>`).
- **Option 2 (Load Package Batch from File):** Reads `batch_input.txt` from disk using `File.ReadAllLines` or `StreamReader` enclosed in a `using` block. Skips corrupted or malformed lines cleanly using `continue`. Evaluates valid lines and adds them to the active manifest in memory.
- **Option 3 (View Active Batch Manifest & Summary):** Iterates through stored manifest records using `for` or `foreach` loops, printing a tabular report followed by the computed batch analytics.
- **Option 4 (Export Audited Manifest to File):** Writes formatted manifest entries and batch summary metrics to `manifest_output.txt` on disk. Alerts the operator with the count of exported records.
- **Option 5 (Exit System):** Displays an operational closing notice and terminates the execution loop cleanly via `break` or loop flag.
- **Input Validation:** If the operator submits an out-of-range option or invalid format, display an error message and reprompt the menu without crashing.

Console Display Specifications (Option 3 Sample)

When Option 3 is executed, the application prints a formatted manifest table with aligned columns:

```
================================================================================================
                                CURRENT BATCH MANIFEST
================================================================================================
Tracking ID  Chg. Weight  Zone  Cargo Type         Total Due (PHP)  Status
------------------------------------------------------------------------------------------------
PKG-1001         8.00 kg     2  General                2,800.00    CLEARED FOR DISPATCH
PKG-1002        15.00 kg     3  Hazardous                 0.00    SHIPMENT REJECTED
PKG-1003         3.00 kg     1  General                 360.00    HELD FOR MANUAL REVIEW
PKG-1004        12.50 kg     2  High-Value Elect.     22,075.00    CLEARED FOR DISPATCH
------------------------------------------------------------------------------------------------
BATCH ANALYTICS:
  Total Processed    : 4 packages
  Cleared Packages   : 2
  Held for Review    : 1
  Rejected Packages  : 1
  Total Revenue      : PHP 25,235.00
  Average Weight     : 9.63 kg
================================================================================================
```

File Storage & Archival Specifications

1. Input Batch File (`batch_input.txt`)

CSV structure:

`TrackingId,Kg,L,W,H,Zone,Type,Value,Invoice,Insurance`

```
PKG-2001,8.0,30,20,20,2,General,8500,true,false
PKG-2002,15.0,50,40,30,3,Hazardous,25000,true,true
PKG-2003,3.0,20,15,10,1,General,4000,false,false
PKG-2004,12.5,40,30,25,2,High-Value Electronics,120000,true,true
```

2. Export Manifest File (`manifest_output.txt`)

Pipe-delimited records generated by Menu Option 4:

```
AUDITED SHIPMENT MANIFEST
Exported: 2026-10-22
-----------------------------------------------
PKG-2001 | 8.00 kg | Zone 2 | PHP 2,800.00 | CLEARED
PKG-2002 | 15.00 kg | Zone 3 | PHP 0.00 | REJECTED
PKG-2003 | 3.00 kg | Zone 1 | PHP 360.00 | HELD
PKG-2004 | 12.50 kg | Zone 2 | PHP 22,075.00 | CLEARED
-----------------------------------------------
SUMMARY: Total=4 | Cleared=2 | Held=1 | Rejected=1
```

Verification & Test Scenarios

Package ID	Input Attributes	Calculated Financials	Operational Status
PKG-2001	8.0 kg | 30×20×20 cm Zone 2 | General PHP 8,500.00 | Inv: Yes | Ins: No	Volumetric: 2.40 kg Chargeable: 8.00 kg Base: PHP 2,800.00 | Duty: PHP 0.00 **Total: PHP 2,800.00**	CLEARED FOR DISPATCH
PKG-2002	15.0 kg | 50×40×30 cm Zone 3 | Hazardous PHP 25,000.00 | Inv: Yes | Ins: Yes	Volumetric: 12.00 kg Chargeable: 15.00 kg *Hazardous prohibited on Zone 3 routes* **Total: PHP 0.00 (Cancelled)**	SHIPMENT REJECTED
PKG-2003	3.0 kg | 20×15×10 cm Zone 1 | General PHP 4,000.00 | Inv: No | Ins: No	Volumetric: 0.60 kg Chargeable: 3.00 kg Base: PHP 360.00 | Duty: PHP 0.00 **Total: PHP 360.00**	HELD FOR MANUAL REVIEW
PKG-2004	12.5 kg | 40×30×25 cm Zone 2 | High-Value Elect. PHP 120,000.00 | Inv: Yes | Ins: Yes	Volumetric: 6.00 kg Chargeable: 12.50 kg Base: PHP 4,375.00 | Surcharge: PHP 1,500.00 Ins: PHP 1,800.00 | Duty: PHP 14,400.00 **Total: PHP 22,075.00**	CLEARED FOR DISPATCH

Grading & Verification Checklist

- ☐ **Modular Methods:** All calculation algorithms, validation logic, and file I/O operations are partitioned into dedicated static methods. No monolithic `Main` method.
- ☐ **Loop Mastery:** `do-while` or `while` powers interactive menu loop; `for` / `foreach` drives manifest rendering; `break` exits cleanly; `continue` skips corrupted input records.
- ☐ **Collection Handling:** In-memory manifest records are dynamically managed using generic `List<T>` collections without fixed-size array overflow issues.
- ☐ **Tuple Return Values:** Multi-value results from weight, fee, and batch aggregation methods are cleanly packaged and returned using C# tuple syntax.
- ☐ **Deterministic Resource Management:** File streams and readers utilize `using` statements or static `File` helper methods to ensure file handles are released immediately.
- ☐ **Logical Namespace Separation:** Classes are organized under `CargoLogistics.Core` (domain logic) and `CargoLogistics.IO` (file & console interfaces).
- ☐ **Zero Compilation Warnings:** Solution compiles cleanly under .NET 8 / 9 with 0 errors and 0 warnings.